

[Dashboard](#) / [My courses](#) / [2023S365114/246/247/248](#) / [Concluding Remarks](#) / [Exam](#)

Started on Monday, 26 June 2023, 1:59 PM

State Finished

Completed on Monday, 26 June 2023, 2:14 PM

Time taken 15 mins 1 sec

Grade 7.00 out of 10.00 (70%)

Question 1

Complete

Mark 0.00 out of 1.00

What is the best possible description for the output of the following snippet of code?

```
@torch.no_grad()
def compute_loss(network: nn.Module, loss_func: nn.Module, x: torch.Tensor, y: torch.Tensor):
    logits = network(x)
    err = loss_func(logits, y)
    err.backward()
    return err
```

Select one:

- ☐ a. A Python `list` object with gradients
- ☐ b. A `torch.Tensor` object without gradients
- ☐ c. A Python `list` object without gradients
- ☒ d. A `torch.Tensor` object with gradients
- ☐ e. A Python `float` object without gradients

Question 2

Complete

Mark 1.00 out of 1.00

What is the most accurate description for the following snippet of code?

```
net = nn.Sequential()
net.add_module("features", FeatureNetwork(...))
net.add_module("classifier", nn.Linear(...))

state_dict = torch.load(...)
net.features.load_state_dict(state_dict)
nn.init.kaiming_uniform_(net.classifier.weight)
nn.init.zeros_(net.classifier.bias)

net.features.requires_grad_(False)
net.classifier.requires_grad_(True)
```

Select one:

- ☐ a. Preparation of a network with pre-trained features for transfer learning (without feature fine-tuning).
- ☐ b. Preparation of a network with pre-trained features for transfer learning with feature fine-tuning.
- ☐ c. Preparation of a fully pre-trained network for transfer learning (without feature fine-tuning).
- ☐ d. Preparation of a fully pre-trained network for transfer learning with feature fine-tuning.
- ☐ e. Preparation of a network that can not be used for transfer learning

Question 3

Complete

Mark 2.00 out of 2.00

Which *two* of the following concepts do **not** belong in the list of answer options? Concepts are said to belong together if they fit within one of the assignments you made this semester.

Select one or more:

- ☐ a. reconstruction
- ☐ b. auto-encoder
- ☒ c. adversarial examples
- ☐ d. reparameterisation trick
- ☒ e. change of variable formula

Question 4

Complete

Mark 0.00 out of 1.00

Which of the following lines of code implements the *ascent* of projected gradient ascent, which is used to compute adversarial examples?

Select one:

- ☐ a. `delta.clamp_(-0.1, 0.1)`
- ☐ b. `logits = network(x + delta)`
- ☒ c. `delta = torch.zeros_like(x, requires_grad=True)`
- ☐ d. `delta += 0.1 * delta.grad.sign()`
- ☐ e. `err.backward()`

Question 5

Complete

Mark 1.00 out of 1.00

The code snippet below aims to produce the **objective to be minimised** for the generator of a GAN. You can assume the label for real (fake) labels to be one (zero). What is wrong with this code?

```
def generator_error(gan: nn.ModuleDict, x: torch.Tensor):  
    mse = nn.MSELoss()  
    z = torch.randn_like(x)  
    with torch.enable_grad():  
        fakes = gan.generator(z)  
    logits = gan.discriminator(fakes)  
    real_labels = torch.ones_like(logits)  
    err = mse(fakes, x)  
    return err
```

Select one:

- ☐ a. The generator will always produce the same outputs during training.
- ☐ b. The generator will be unable to receive weight updates.
- ☐ c. The generator and discriminator have been swapped.
- ☐ d. The generator weight updates will have the wrong sign.
- ☒ e. The generator error is computed with the wrong kind of loss function.

Question 6

Complete

Mark 1.00 out of 1.00

How would you implement the inverse of the forward pass for an additive coupling layer?

☒ a.

```
def coupling(y):  
    y2, y1 = torch.chunk(y, 2, dim=1)  
    shift = self.conditioner(y2)  
    x1, x2 = y1 - shift, y2  
    return torch.cat([x1, x2], dim=1)
```

☐ b.

```
def coupling(y):  
    y1, y2 = torch.chunk(y, 2, dim=1)  
    scale = self.conditioner(y2)  
    x1, x2 = y1 / scale, y2  
    return torch.cat([x1, x2], dim=1)
```

☐ c.

```
def coupling(x):  
    x1, x2 = torch.chunk(x, 2, dim=1)  
    scale = self.conditioner(x2)  
    y1, y2 = x1 * scale, x2  
    return torch.cat([y1, y2], dim=1)
```

☐ d.

```
def coupling(x):  
    x1, x2 = torch.chunk(x, 2, dim=1)  
    shift = self.conditioner(x2)  
    y1, y2 = x1 + shift, x2  
    return torch.cat([y1, y2], dim=1)
```

☐ e.

```
def coupling(x):  
    x1, x2 = torch.chunk(x, 2, dim=1)  
    shift = self.conditioner(x2)  
    y1, y2 = x1 + shift, x2  
    return torch.cat([y2, y1], dim=1)
```

Question 7

Complete

Mark 2.00 out of 3.00

Match the code snippets to the corresponding ML research papers.

Hint: Try to find algorithms, figures or equations in the papers, there should be no need to read them.

Click on the selected ML research paper to view it:

- ▶ **Explaining and Harnessing Adversarial Examples**
- ▶ **Adversarial examples in the physical world**
- ▶ **Adversarial Examples Are Not Easily Detected: Bypassing Ten Detection Methods**

```
import torch
from torch import nn

def adversarial_attack(x:
    torch.Tensor, y: torch.Tensor,
    network: nn.Module,
                        epsilon:
float = .1, eta: float = .1):
    ce =
nn.CrossEntropyLoss(reduction="sum
")
    delta = torch.zeros_like(x
```

Explaining and Harnessing Adversarial Examples

```
import torch
from torch import nn

def adversarial_attack(x:
    torch.Tensor, y: torch.Tensor,
    network: nn.Module,
                        epsilon:
float = .1, eta: float = .1,
steps: int = 10):
    mr =
nn.MarginRankingLoss(reduction="su
m")
```

Choose...

```
import torch
from torch import nn

def adversarial_attack(x:
    torch.Tensor, y: torch.Tensor,
    network: nn.Module,
                        epsilon:
float = .1, eta: float = .1,
steps: int = 10):
    ce =
nn.CrossEntropyLoss(reduction="sum
")
```

Adversarial examples in the physical world

◀ Final Question

Jump to...